# Computer System Performance Comparisons

# Comparing Computer Performance

- How to compare performance of different machines?
- What factors affect performance?
- How to improve performance?

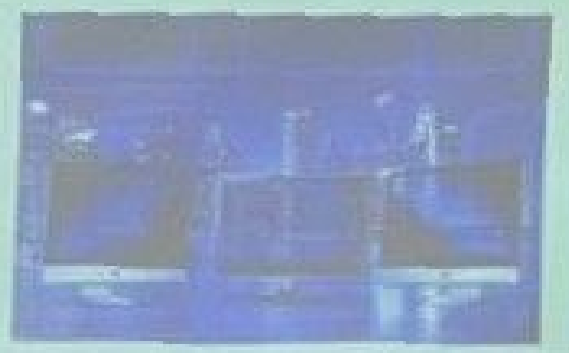
# Comparing Computer Performance

- To compare the performance of two machines (or CPUs) "A", "B" running a given specific program:

$$\text{Performance}_A \approx 1 \;/\; \text{Execution Time}_A$$
$$\text{Performance}_B \approx 1 \;/\; \text{Execution Time}_B$$

- *Machine A is n times faster than machine B means* (or slower? if n < 1)

$$\boxed{\textit{Speedup} = n = \frac{\text{Performance}_A}{\text{Performance}_B} = \frac{\text{Execution Time}_B}{\text{Execution Time}_A}}$$



# Comparing Computer Performance: Example

- For a given program:

  Execution time on machine A: $\text{Execution}_A = 1$ second

  Execution time on machine B: $\text{Execution}_B = 10$ seconds

  Speedup = $\text{Performance}_A \;/\; \text{Performance}_B$

  $= \text{Execution Time}_B \;/\; \text{Execution Time}_A$

  $= 10 / 1 = 10$

- The performance of machine A is 10 times the performance of machine B when running this program, or: Machine A is said to be 10 times faster than machine B when running this program.

# Amdahl's Law

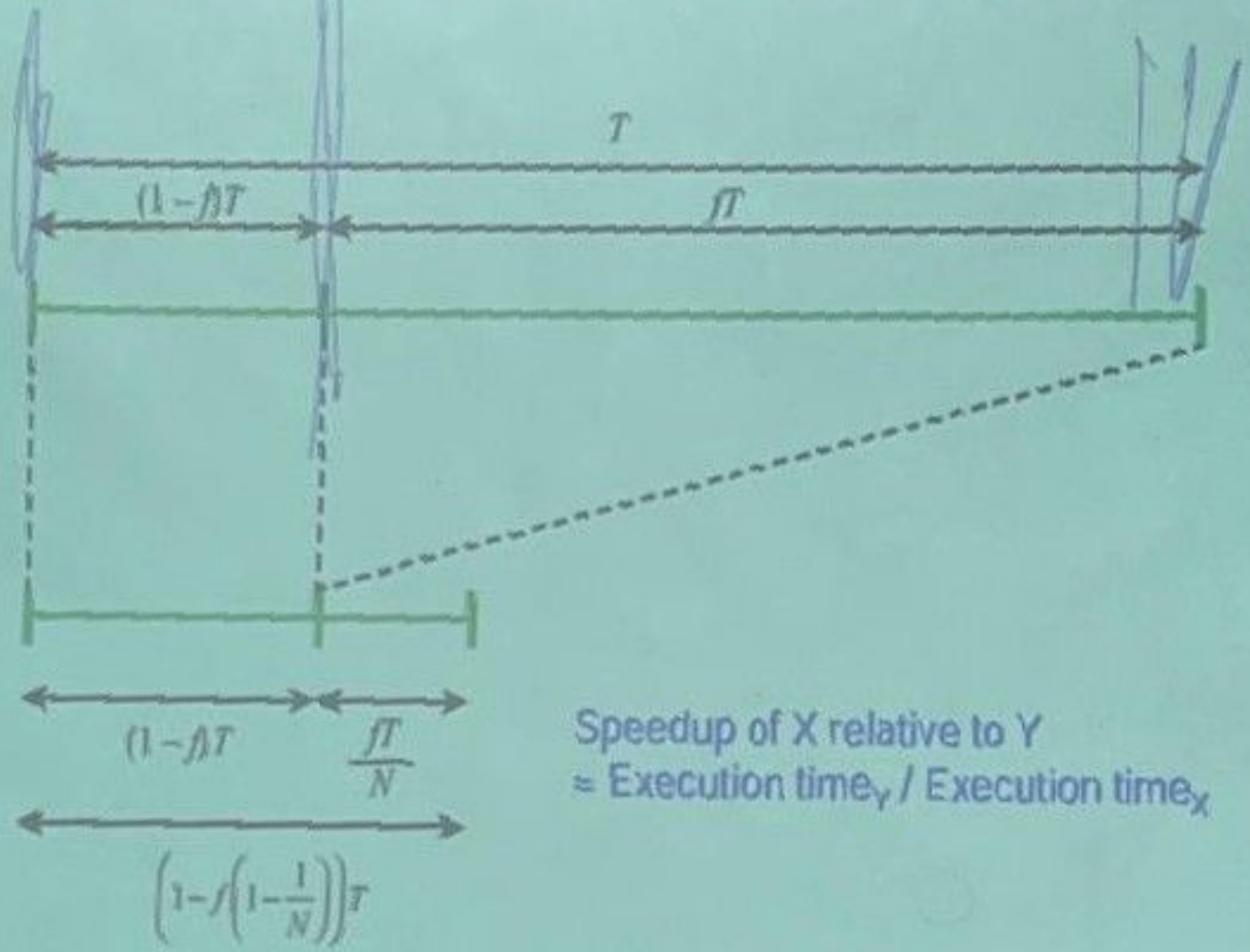
- Consider a program running on a single processor such that a fraction (1 - f) of the execution time involves code that is inherently sequential, and a fraction f that involves code that is infinitely parallelizable with no scheduling overhead.
- Let T be the total execution time of the program using a single processor.
- Then the speedup using a parallel processor with N processors that fully exploits the parallel portion of the program is as follows:

$$\text{Speedup} = \frac{\text{Time to execute program on a single processor}}{\text{Time to execute program on } N \text{ parallel processors}}$$

$$= \frac{T(1-f) + Tf}{T(1-f) + \frac{Tf}{N}} = \frac{1}{(1-f) + \frac{f}{N}}$$

amdahl

# Illustration of Amdahl's Law




Speedup of X relative to Y
≈ Execution time$_Y$ / Execution time$_X$

# Amdahl's Law

- *Computer system designers look for ways to improve system performance by advances in technology or change in design.*
- *Examples include the use of parallel processors, the use of a memory cache hierarchy, and speedup in memory access time and I/O transfer rate due to technology improvements.*
- *In all of these cases, it is important to note that a speedup in one aspect of the technology or design does not result in a corresponding improvement in performance.*
- This limitation is succinctly expressed by Amdahl's law.
- Amdahl's law was first proposed by Gene Amdahl in 1967.
- *The law deals with the potential speedup of a program using multiple processors compared to a single processor.*
- Nevertheless, Amdahl's law illustrates the problems facing industry in the development of multicore machines with an ever-growing number of cores;
- The software that runs on such machines must be adapted to a highly parallel execution environment to [illegible] the power of parallel processing.

# Amdahl's Law

- Amdahl's law can be generalized to evaluate any design or technical improvement in a computer system.
- Consider any enhancement to a feature of a system that results in a speedup.
- The speedup can be expressed as

$$\text{Speedup} = \frac{\text{Performance after enhancement}}{\text{Performance before enhancement}}$$

$$= \frac{\text{Execution time before enhancement}}{\text{Execution time after enhancement}}$$

# Amdahl's Law (Fixed Size Speedup)

- Let $f$ be the fraction of a program that is sequential
  - $1-f$ is the fraction that can be parallelized
- Let $T_1$ be the execution time on 1 processor
- Let $T_p$ be the execution time on p processors
- $S_p$ is the speedup

$$S_p = T_1 / T_p$$
$$= T_1 / (fT_1 + (1-f)T_1/p)$$
$$= 1 / (f + (1-f)/p)$$

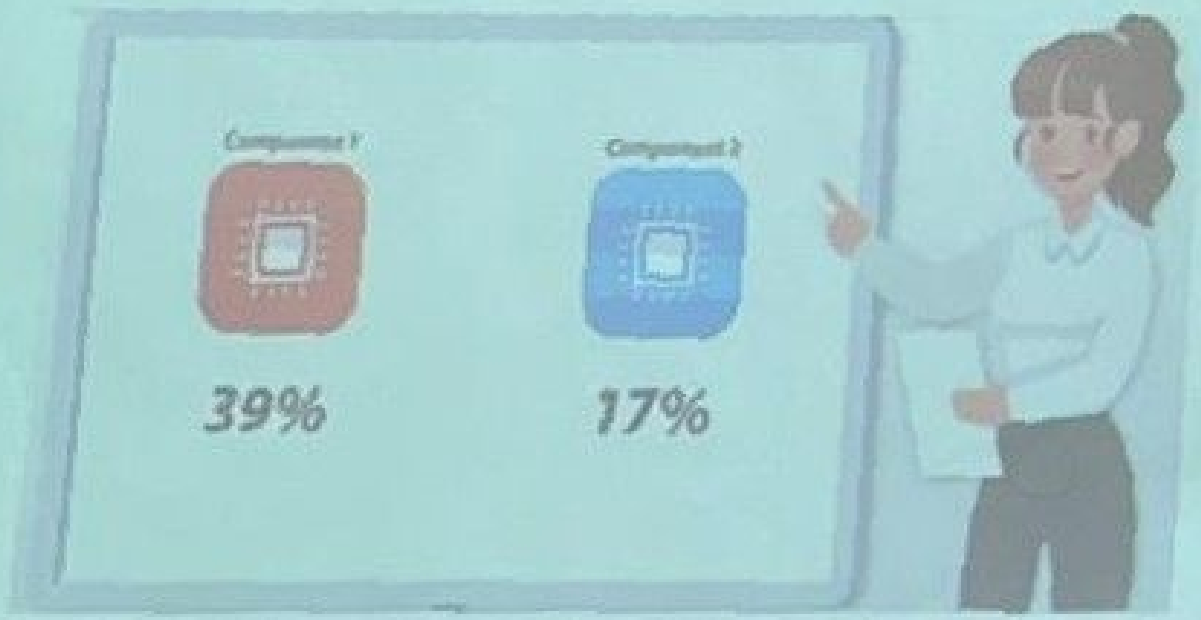
- As $p \to \infty$

$$1/f$$

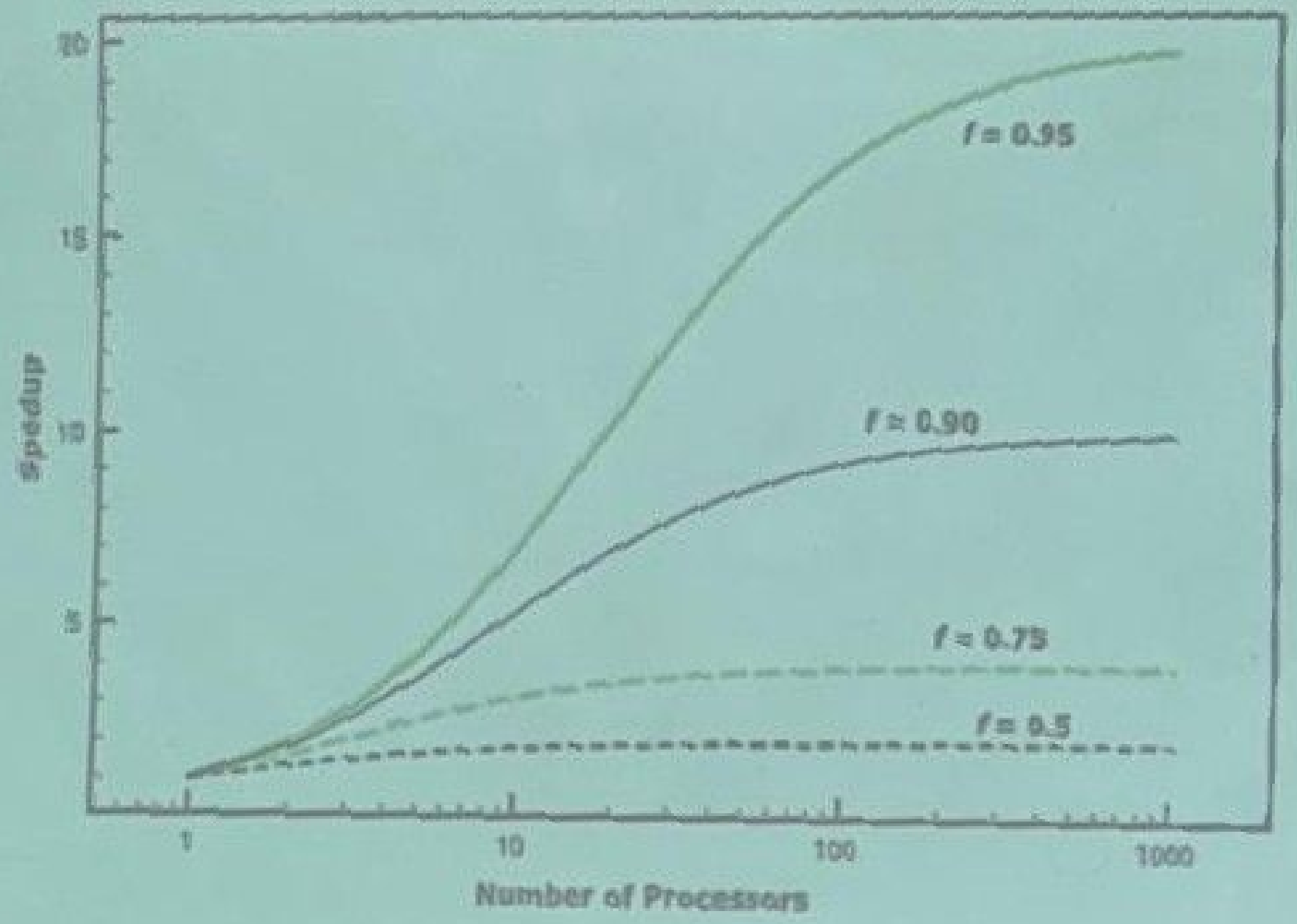
$$S_p \to 1/f$$

# Amdahl's Law: Example

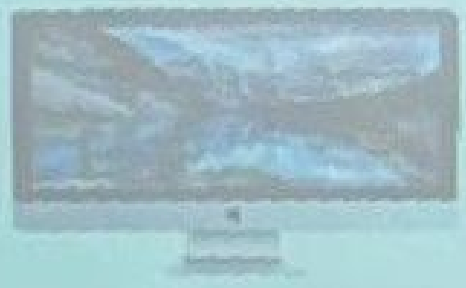
- Suppose that a task makes extensive use of floating-point operations, with 40% of the time consumed by floating-point operations. With a new hardware design, the floating-point module is sped up by a factor of K.
- Then the overall **speedup is 1.67**



# Illustration of Amdahl's Law



# Comparison of Computers

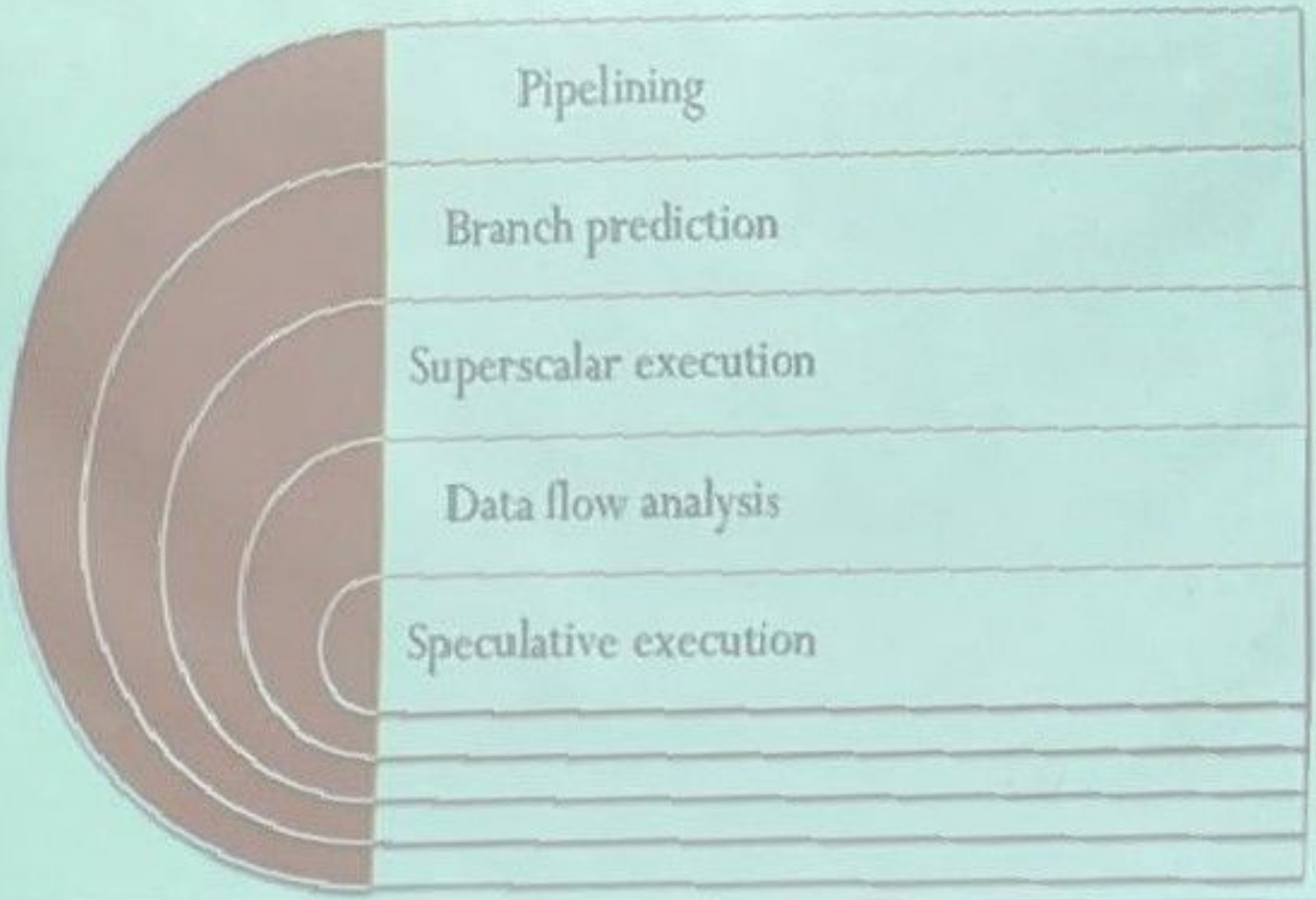
| |  | | |
|---|---|---|---|
| Computer | Apple 27" iMac Retina 5K | Digital Storm Apollo | DIY Custom Build |
| Processor | 4.0 GHz Intel Core i7 Quad-Core | 3.6 GHz Intel Core i7-6850K 6-Core | 3.2 GHz Intel Core i7-6900K 8-Core |
| RAM | 32GB (DDR3) | 32GB (DDR4) | 64GB (DDR4) |
| Hard Drive | 2TB Fusion Drive | 500GB SSD + 2TB HDD | 1TB SSD + 2TB HDD |
| GPU | AMD Radeon R9 M390 (2GB Memory) | NVIDIA GTX 1070 (8GB Memory) | Asus GForce GTX 1080 (8GB Memory) |
| Warranty | 1 yr Standard / AppleCare | 3 Years Labor, 1 Year Parts | Varies by Component |
| Cinebench Score | 900 | 1300 | 1500 |
| Price | $3099 | $2835 | $3146 |

# Improvements in Chip Organization and Architecture

- *Increase hardware speed of processor*
  - Fundamentally due to shrinking logic gate size
    - More gates, packed more tightly, increasing clock rate
    - Propagation time for signals reduced
- *Increase size and speed of caches*
  - Dedicating part of processor chip
    - Cache access times drop significantly
- *Change processor organization and architecture*
  - Increase effective speed of instruction execution
  - Parallelism

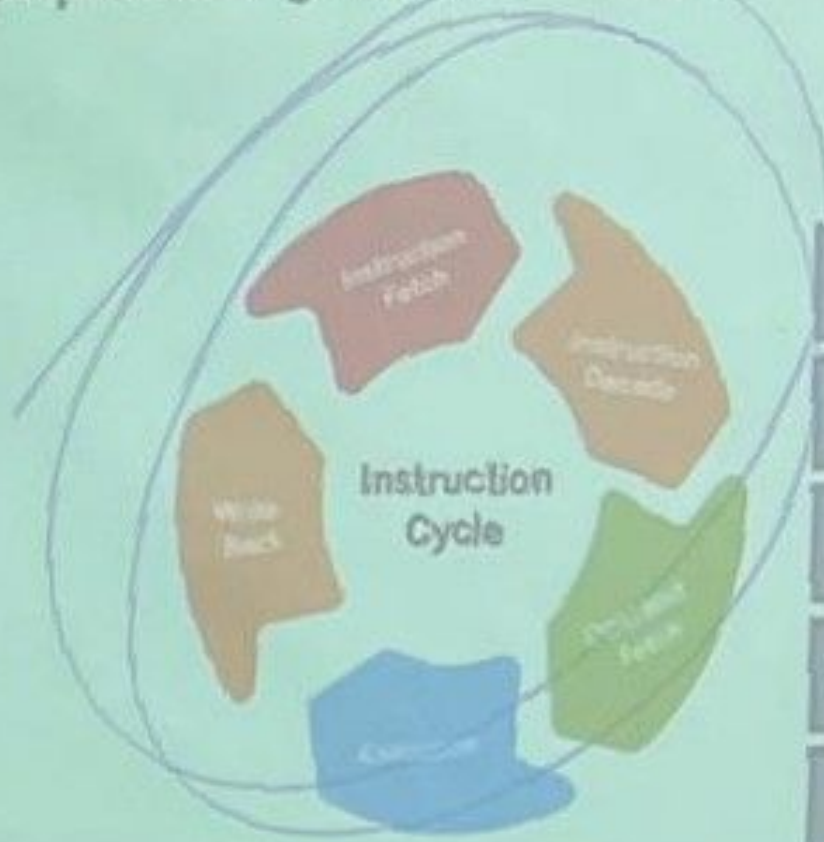
# Microprocessor Speed

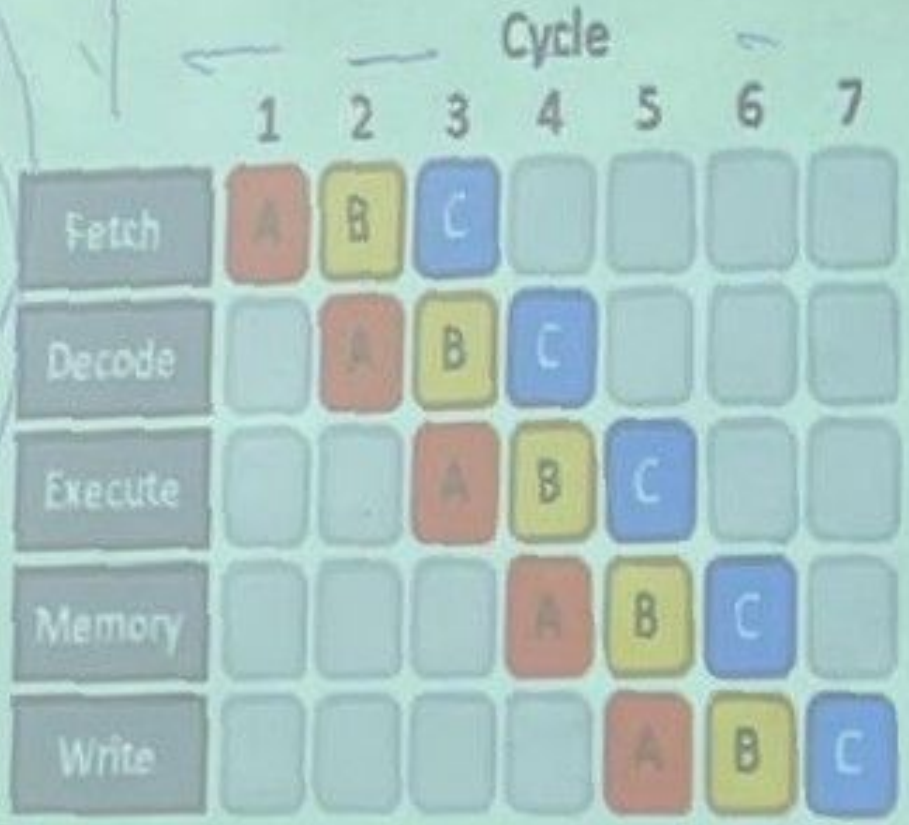
Techniques built into contemporary processors include:



# Pipelining

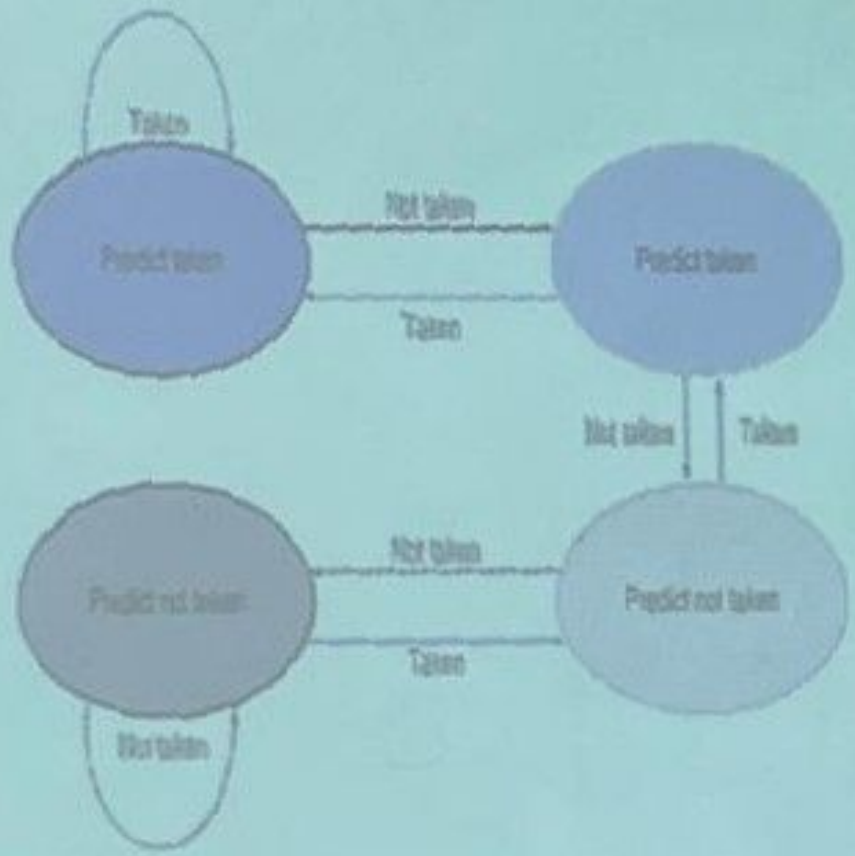
Processor moves data or instructions into a conceptual pipe with all stages of the pipe processing simultaneously



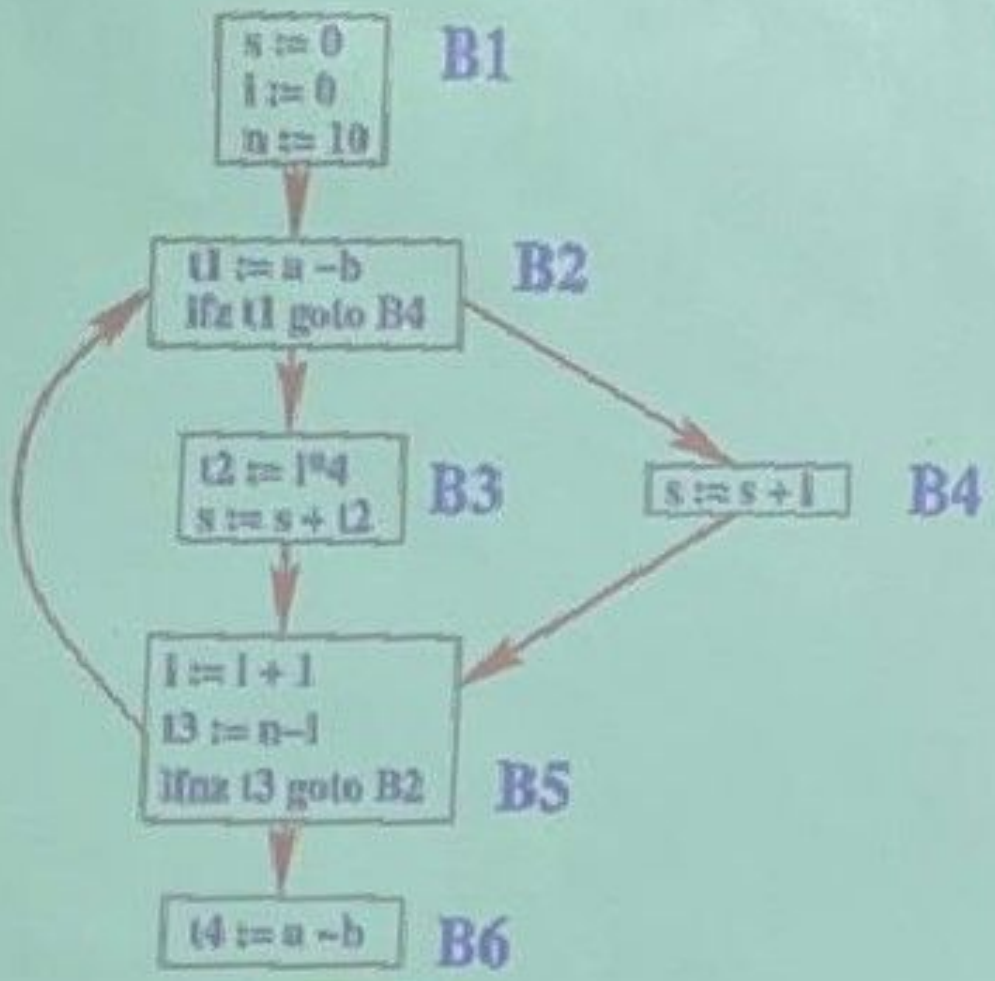



# Branch Prediction

*Processor looks ahead in the instruction code fetched from memory and predicts which branches, or groups of instructions, are likely to be processed next.*



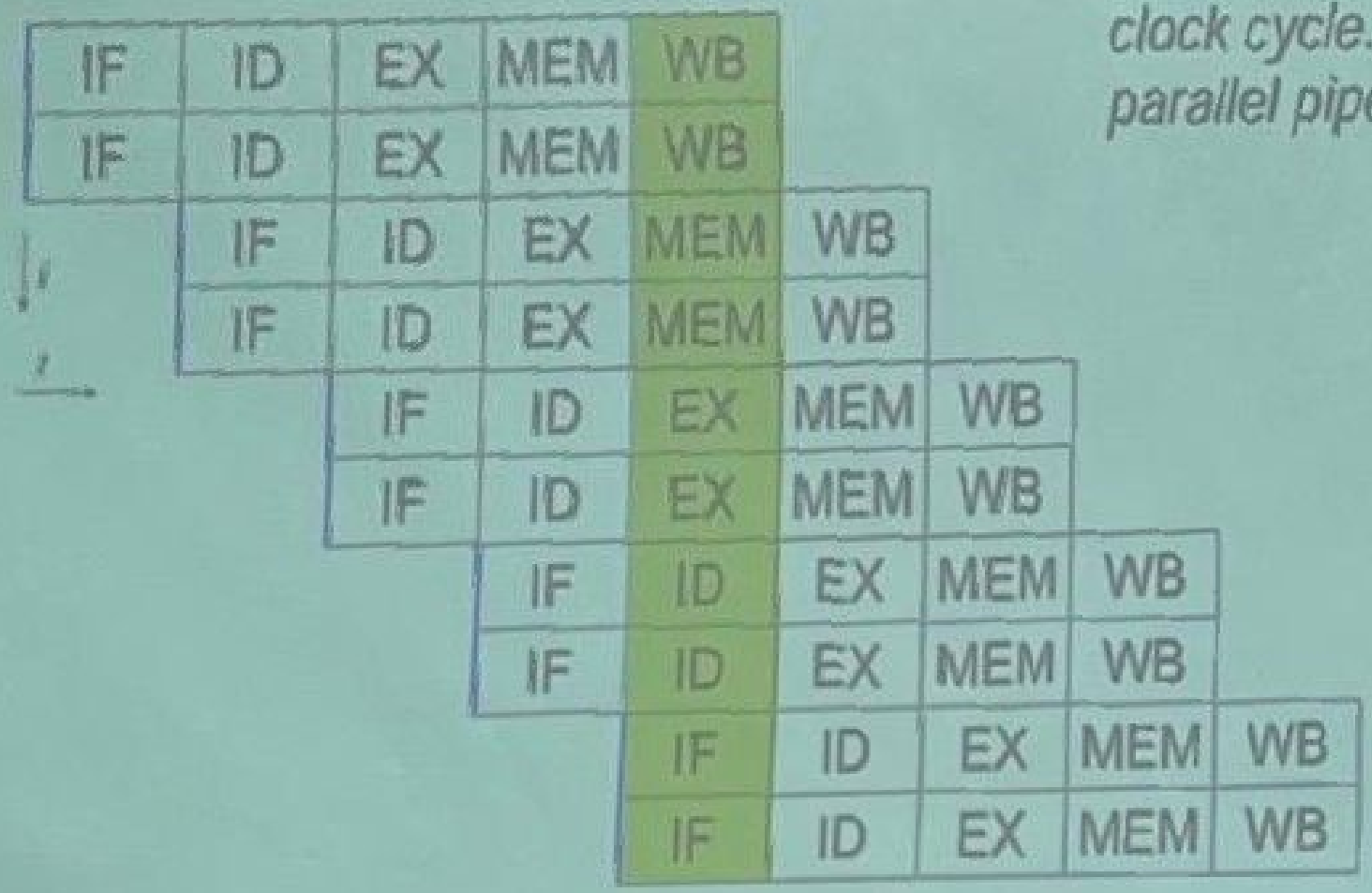
# Data Flow Analysis



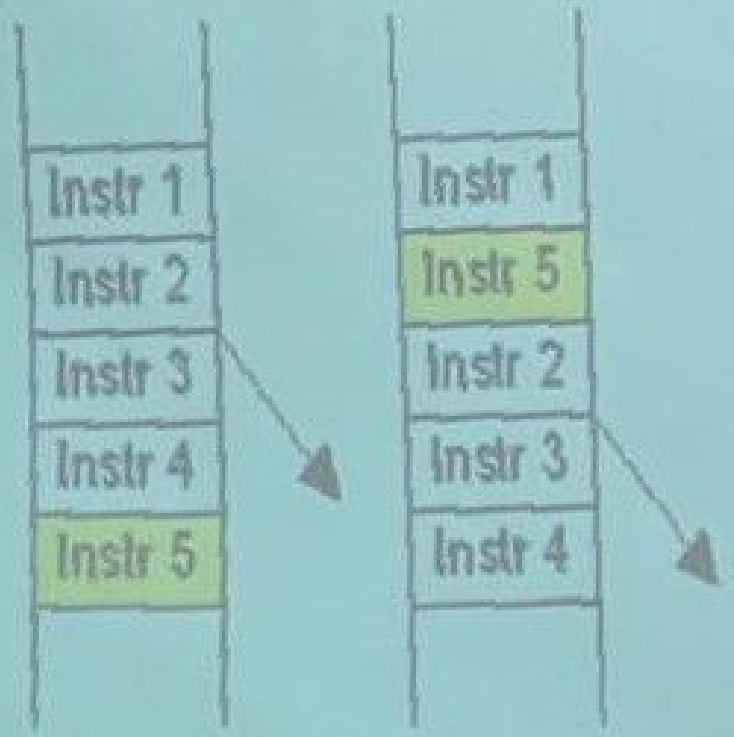

*Processor analyzes which instructions are dependent on each other's results, or data, to create an optimized schedule of instructions.*

# Superscalar Execution

*This is the ability to issue more than one instruction in every processor clock cycle. (In effect, multiple parallel pipelines are used.)*



# Speculative Execution



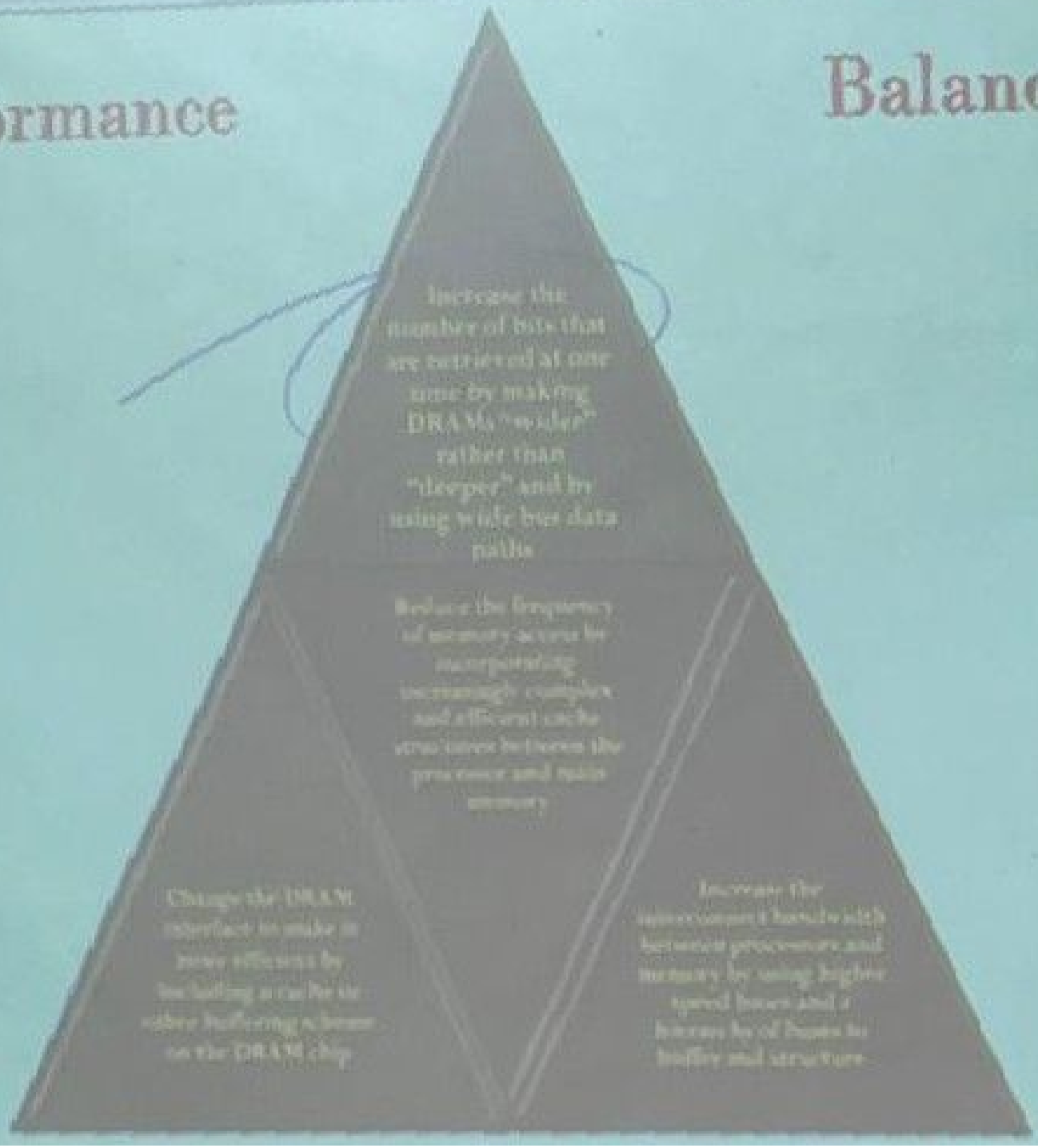

*Using branch prediction and data flow analysis, some processors speculatively execute instructions ahead of their actual appearance in the program execution, holding the results in temporary locations, keeping execution engines as busy as possible*

# Performance Balance

- While processor power has raced ahead at breakneck speed, other critical components of the computer have not kept up.
- The result is a need to look for performance balance: an adjusting of the organization and architecture to compensate for the mismatch among the capabilities of the various components.
- The problem created by such mismatches is particularly critical at the interface between processor and main memory.
- While processor speed has grown rapidly, the speed with which data can be transferred between main memory and the processor has lagged badly.
- The interface between processor and main memory is the most crucial pathway in the entire computer because it is responsible for carrying a constant flow of program instructions and data between memory chips and the processor.
- If memory or the pathway fails to keep pace with the processor's insistent demands, the processor stalls in a wait state, and valuable processing time is lost.

# Performance Balance

- A system architect can attack this problem in a number of ways, all of which are reflected in contemporary computer designs. Consider the following examples:
  - Increase the number of bits that are retrieved at one time by making DRAMs "wider" rather than "deeper" and by using wide bus data paths.
  - Change the DRAM interface to make it more efficient by including a cache or other buffering scheme on the DRAM chip.
  - Reduce the frequency of memory access by incorporating increasingly complex and efficient cache structures between the processor and main memory.
    - This includes the incorporation of one or more caches on the processor chip as well as on an off-chip cache close to the processor chip.
  - Increase the interconnect bandwidth between processors and memory by using higher-speed buses and a hierarchy of buses to buffer and structure data flow.

# Performance

# Balance